

Результаты оптимизации на реальных данных (1.8 млн строк)

Датасет: NASA Access Logs (июль 1995)

Источник: Публичный датасет, предоставленный NASA и обработанный в рамках учебного проекта. Ссылка на оригинальный репозиторий:

[diljyotsingh019/NASA-access-logs-on-Big-data: Processing NASA's access log files from 1995 using HADOOP](https://github.com/diljyotsingh019/NASA-access-logs-on-Big-data: Processing NASA's access log files from 1995 using HADOOP)

Структура данных: Логи HTTP-запросов к серверам NASA. Каждая строка содержит IP-адрес, временную метку, метод запроса, URL, протокол, статус-код и размер ответа.

Исходный датасет содержал 1 886 838 строк логов NASA. В ходе подготовки выполнены следующие шаги:

1. Удаление дубликатов

Удалены полностью идентичные строки путем создания новой таблицы с группировкой, выбрав наименьший id из всех строк в группе, и удаления старой версии.

```
CREATE TABLE nasa_logs_dedup AS
SELECT MIN(id) as id, ip, log_date, log_time, method, url, protocol,
status, bytes
FROM nasa_logs
GROUP BY ip, log_date, log_time, method, url, protocol, status, bytes;
```

Результат: Осталось 1 886 006 строк (удалено 832 дубликата).

2. Нормализация URL

URL приведены к нижнему регистру для корректной группировки.

```
UPDATE nasa_logs SET url_normalized = LOWER(url);
```

Результат: Обнаружено и объединено 290 вариантов URL, различающихся только регистром.

3. Валидация IP-адресов

Добавлена колонка `is\_valid\_ip` для разделения числовых IPv4 и доменных имен.

- 1) `ALTER TABLE nasa_logs ADD COLUMN is_valid_ip INTEGER DEFAULT 0;`
`UPDATE nasa_logs SET is_valid_ip = 1`
`WHERE ip LIKE '%.%.%.%'`
`AND ip NOT GLOB '*[a-zA-Z]*'`
`AND ip != ''`
`AND LENGTH(ip) >= 7;`
- 2) `CREATE INDEX idx_valid_ip ON nasa_logs(is_valid_ip);`

Для гео-аналитики нужно использовать только строки с `is\_valid\_ip = 1`

Результат: 22.2% записей – валидные IPv4, 77.8% – доменные имена.

Итог: Создана чистая, размеченная база данных, готовая для оптимизации запросов и анализа.

Эксперимент №1

Был выполнен поиск записей по IP-адресу (`199.72.81.55`) до и после создания индекса `idx\_ip` на колонке `ip`:

```
SELECT * FROM nasa_logs
WHERE ip = '199.72.81.55';
```

Result: 2611 мс для 122 строк.

Создали индекс:

```
CREATE INDEX idx_ip ON nasa_logs(ip);
```

После создания индекса поиск по ip = '199.72.81.55' занял:
 122 строк возвращено за 117 мс.
 Ускорение в ~22.3 раза.

	id	ip	log_date	log_time	method
1	1	199.72.81.55	1995-Jul-01	00:00:01	GET
2	42	199.72.81.55	1995-Jul-01	00:00:59	GET
3	76	199.72.81.55	1995-Jul-01	00:01:43	GET
4	79	199.72.81.55	1995-Jul-01	00:01:46	GET
5	84	199.72.81.55	1995-Jul-01	00:01:50	GET
6	85	199.72.81.55	1995-Jul-01	00:01:51	GET

Execution finished without errors.
 Result: 122 строк возвращено за 2611мс
 At line 1:
 SELECT * FROM nasa_logs WHERE ip = '199.72.81.55';

Execution finished without errors.
 Result: 122 строк возвращено за 117мс
 At line 1:
 SELECT * FROM nasa_logs WHERE ip = '199.72.81.55';

Эксперимент №2

Поиск по диапазону дат:

```
SELECT COUNT(*) FROM nasa_logs
WHERE log_date BETWEEN '1995-Jul-15' AND
'1995-Jul-31';
```

Result: 1 строк возвращено за 1642 мс

```
Execution finished without errors.
Result: 1 строк возвращено за 1642мс
At line 1:
SELECT COUNT(*) FROM nasa_logs
WHERE log_date BETWEEN '1995-Jul-15' AND '1995-Jul-31';
```

Создали индекс:

```
CREATE INDEX idx_log_date ON
nasa_logs(log_date);
```

После создания индекса поиск занял:

177 мс.

Ускорение в ~9.3 раза.

```
Execution finished without errors.
Result: 1 строк возвращено за 177мс
At line 1:
SELECT COUNT(*) FROM nasa_logs
WHERE log_date BETWEEN '1995-Jul-15' AND '1995-Jul-31';
```

Эксперимент №3

Топ-50 IP с фильтром по статусу:

```
SELECT ip, status, COUNT(*) as cnt
FROM nasa_logs
WHERE status = 404
GROUP BY ip
ORDER BY cnt DESC
LIMIT 50;
```

Result: 1 строк возвращено за 1607 мс

```
Execution finished without errors.
Result: 50 строк возвращено за 1607мс
At line 1:
SELECT ip, status, COUNT(*) as cnt
FROM nasa_logs
WHERE status = 404
GROUP BY ip
ORDER BY cnt DESC
LIMIT 50;
```

```
CREATE INDEX idx_status ON
nasa_logs(status);
```

После создания индекса поиск занял:

50 строк возвращено за 136 мс

Ускорение в ~11.8 раза.

! Инсайт:

`idx_status + idx_ip` | 244 мс | Дополнительный индекс не дал выигрыша

```
Execution finished without errors.
Result: 50 строк возвращено за 136мс
At line 1:
SELECT ip, status, COUNT(*) as cnt
FROM nasa_logs
WHERE status = 404
GROUP BY ip
ORDER BY cnt DESC
LIMIT 50;
```

Выводы:

1. Индексы критически важны для производительности на больших объемах данных (1.8 млн строк).
2. Каждый индекс должен соответствовать паттерну запроса. Универсальных индексов не существует.
3. Подготовка данных – обязательный этап перед анализом.
4. В реальных данных всегда есть особенности. В логах NASA 78% записей в поле ``ip`` оказались доменными именами, что потребовало дополнительной обработки.

Результаты проекта:

- Подготовлено 1.8 млн строк логов NASA (удалены дубликаты, нормализованы URL, валидированы IP)
- Оптимизированы SQL-запросы с помощью индексов:
ускорение до 11.8 раз (с 1607 мс до 136 мс)
- Доказано на практике: индексы эффективны только для соответствующих запросов (лишний индекс не дал выигрыша)
- Создан воспроизводимый пайплайн очистки и оптимизации данных